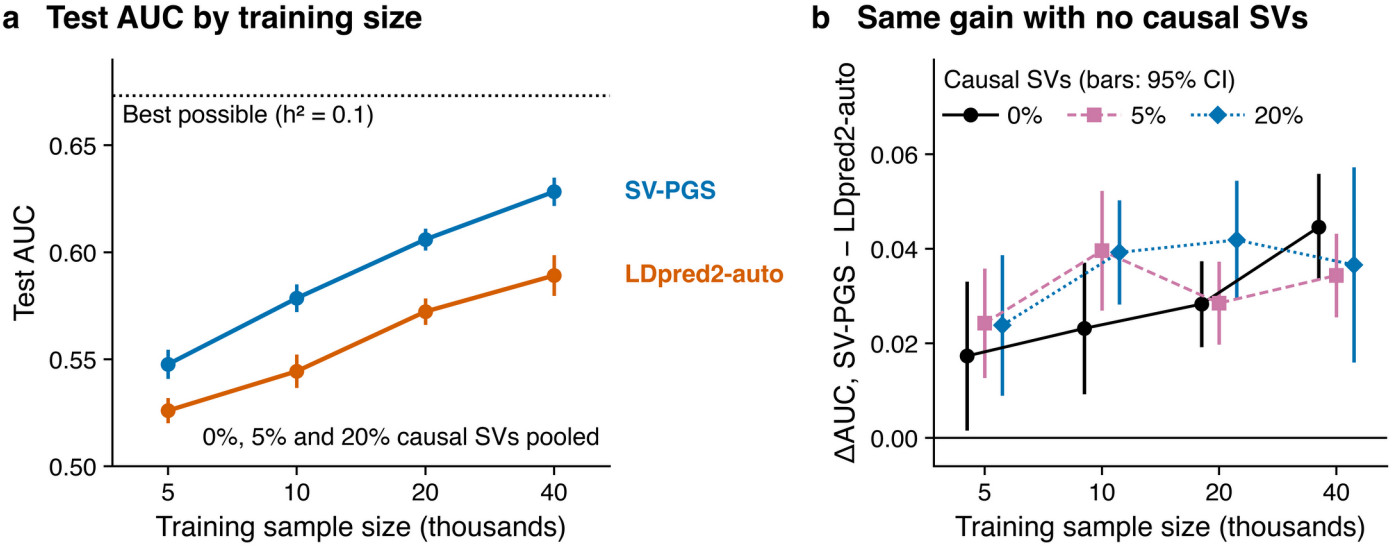


Figure 1 | SV-PGS is more accurate than LDpred2-auto at every training size, by a similar margin whether 0%, 5% or 20% of the causal variants are SVs, so the gain does not come from causal SVs. A simulated binary trait on chromosome 22 (liability h^2 = 0.1, prevalence 10%, 200 causal variants) scored in the same 10,000 held-out individuals. SV-PGS fits every SNV, indel and imputed SV, and LDpred2-auto HapMap3-density SNVs of the same training individuals. a, Test AUC, mean and Student-t 95% interval over the three settings' simulations pooled (42, 37, 38, 16 at 5k-40k). Dotted line: the largest AUC any score can reach, that of the true genetic value at h^2 = 0.1 (Wray et al. 2010). b, Paired difference in AUC on the same held-out individuals, per setting, mean and Student-t 95% interval.

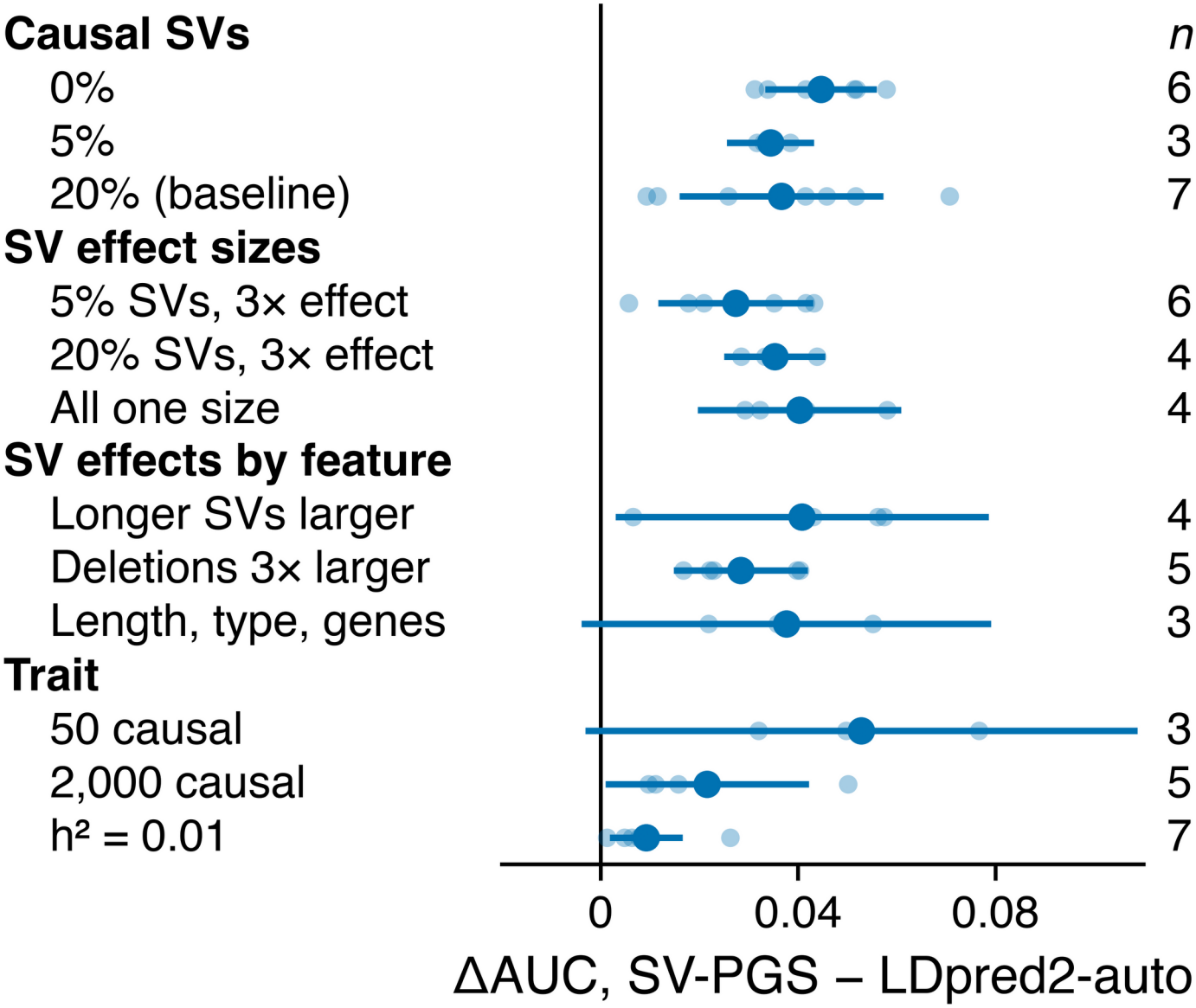


Figure 2 | SV-PGS is more accurate than LDpred2-auto across simulated genetic architectures. Paired difference in test AUC on the same 10,000 held-out individuals between SV-PGS (every SNV, indel and imputed SV) and LDpred2-auto (HapMap3-density SNVs), both trained on the same 40,000 individuals: each simulation (light blue), mean and Student-t 95% interval (blue, with an arrowhead where one runs past 0.11), n simulations at right. SV-PGS had the higher AUC in 57 of 57 simulations. The baseline architecture has liability $h^2 = 0.1$, prevalence 10%, 200 causal variants and 20% causal SVs with SV effects drawn like small-variant effects. Each other architecture changes one or more of these settings or how SV effects are drawn: '3×' triples the SV effect SD, 'a few very large' draws SV effects from a Student t with 3 degrees of freedom, 'all one size' gives every causal SV the same effect size with a random sign, 'well-imputed' draws causal SVs only from those imputed with $r^2 \geq 0.8$, and 'common' only from SVs of frequency ≥ 5%. Fewer than three simulations at 40,000: a few very large SV effects, only well-imputed SVs, only well-imputed SVs, 3× effect, only common SVs, prevalence 1%.

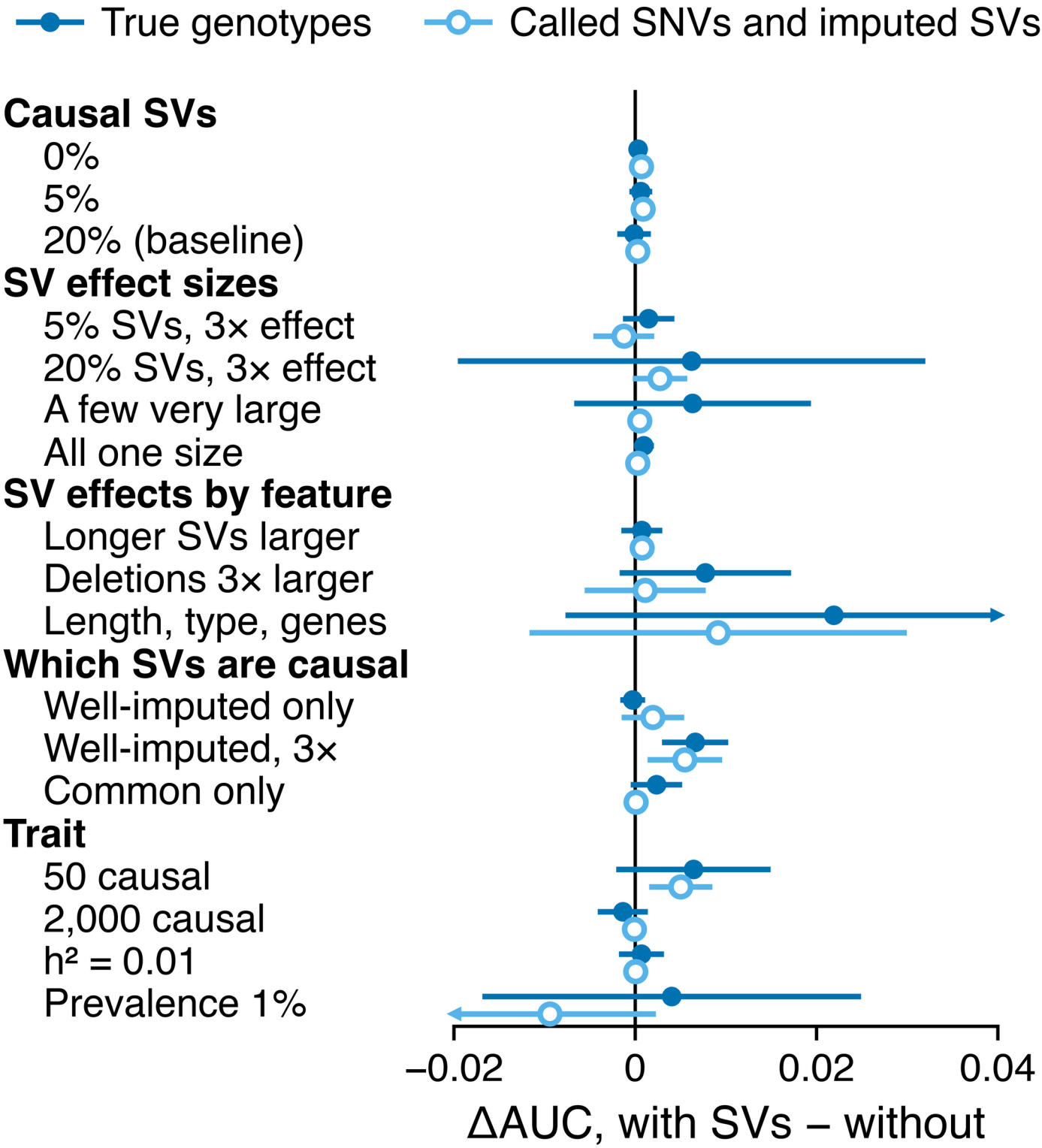


Figure 3 | Adding SVs changed test AUC little in each architecture. With true genotypes the gain's interval lies above zero for 1 of 17 architectures: 'Well-imputed, 3×'. Paired difference in test AUC on the same held-out individuals between SV-PGS fitted with and without SVs on one 3 Mb chromosome 22 window (60,000 records), with true genotypes at every variant (filled) or with SNVs and indels called from simulated reads and SVs imputed by Beagle (open). Each simulation's difference is its mean over the training sizes both genotype sources finished, from 5,000 to 40,000 (Figure 4 shows the sizes apart). Mean and Student-t 95% interval over 5-10 simulations per architecture, an arrowhead marking an interval that runs off the axis. Both sources use the same simulated individuals and causal variants. The widest true-genotype interval, 'Length, type, genes', comes from one simulation at 0.051. The baseline architecture has liability $h^2 = 0.1$, prevalence 10%, 200 causal variants and 20% causal SVs with SV effects drawn like small-variant effects. Each other architecture changes one or more of these settings or how SV effects are drawn: '3×' triples the SV effect SD, 'a few very large' draws SV effects from a Student t with 3 degrees of freedom, 'all one size' gives every causal SV the same effect size with a random sign, 'well-imputed' draws causal SVs only from those imputed with $r^2 \geq 0.8$, and 'common' only from SVs of frequency ≥ 5%.

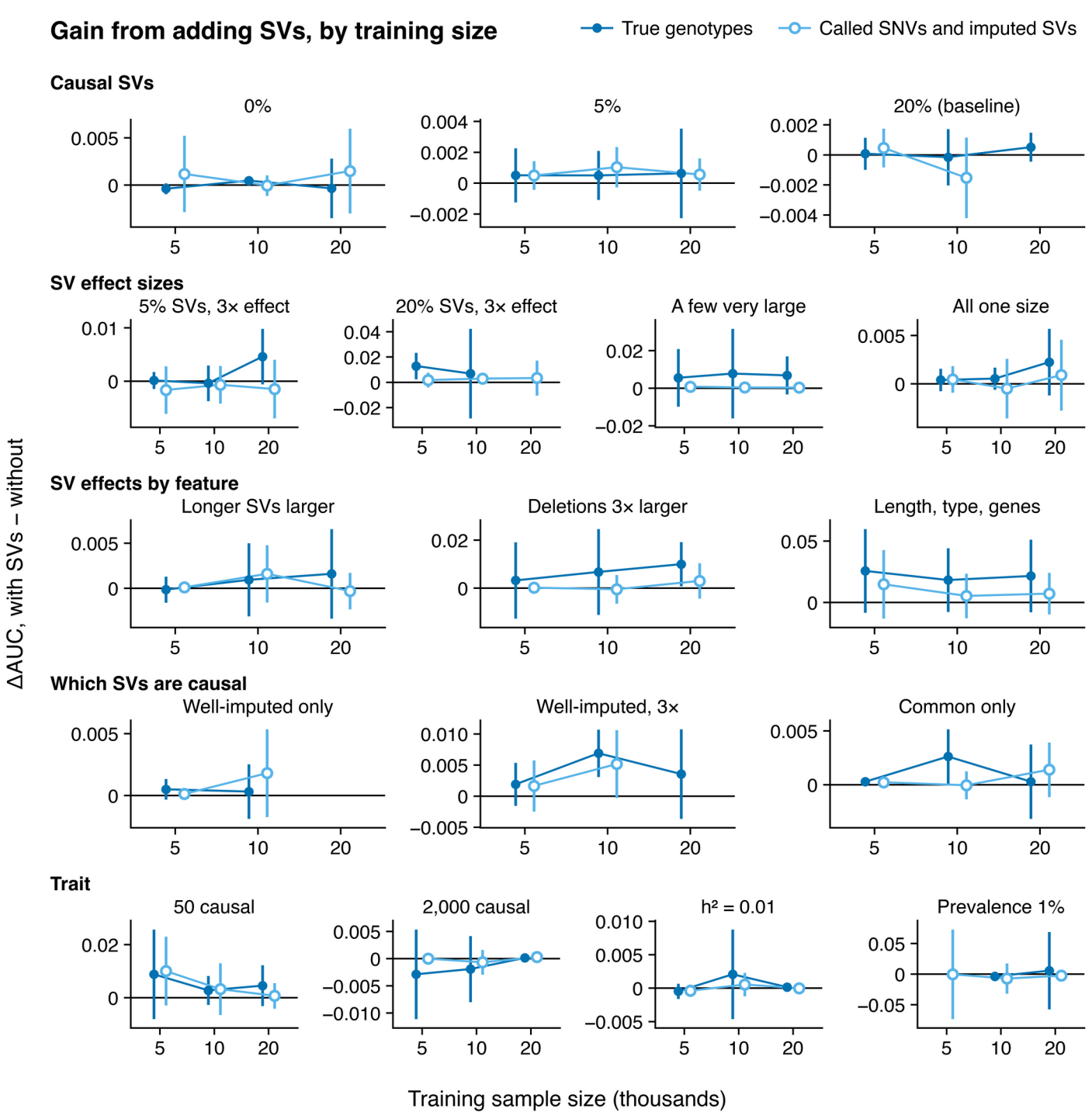


Figure 4 | The gain from adding SVs at each training size from 5,000 to 20,000, for each simulated architecture. Each panel shows the paired difference in test AUC between SV-PGS fitted with and without SVs on one 3 Mb chromosome 22 window, with true genotypes at every variant (filled) or with SNVs called from simulated reads and SVs imputed by Beagle (open): the mean over simulations and its Student-t 95% interval. Each panel has its own y-axis scale. A mean is drawn when its training size has at least three simulations. Each simulation uses the same causal variants and the same test individuals at every training size. We fitted 0-3 simulations per architecture and genotype source at 40,000 training individuals, too few to show. Figure 3 averages them into each simulation's gain. The baseline architecture has liability $h^2 = 0.1$, prevalence 10%, 200 causal variants and 20% causal SVs with SV effects drawn like small-variant effects. Each other architecture changes one or more of these settings or how SV effects are drawn: '3×' triples the SV effect SD, 'a few very large' draws SV effects from a Student t with 3 degrees of freedom, 'all one size' gives every causal SV the same effect size with a random sign, 'well-imputed' draws causal SVs only from those imputed with $r^2 \geq 0.8$, and 'common' only from SVs of frequency ≥ 5%.

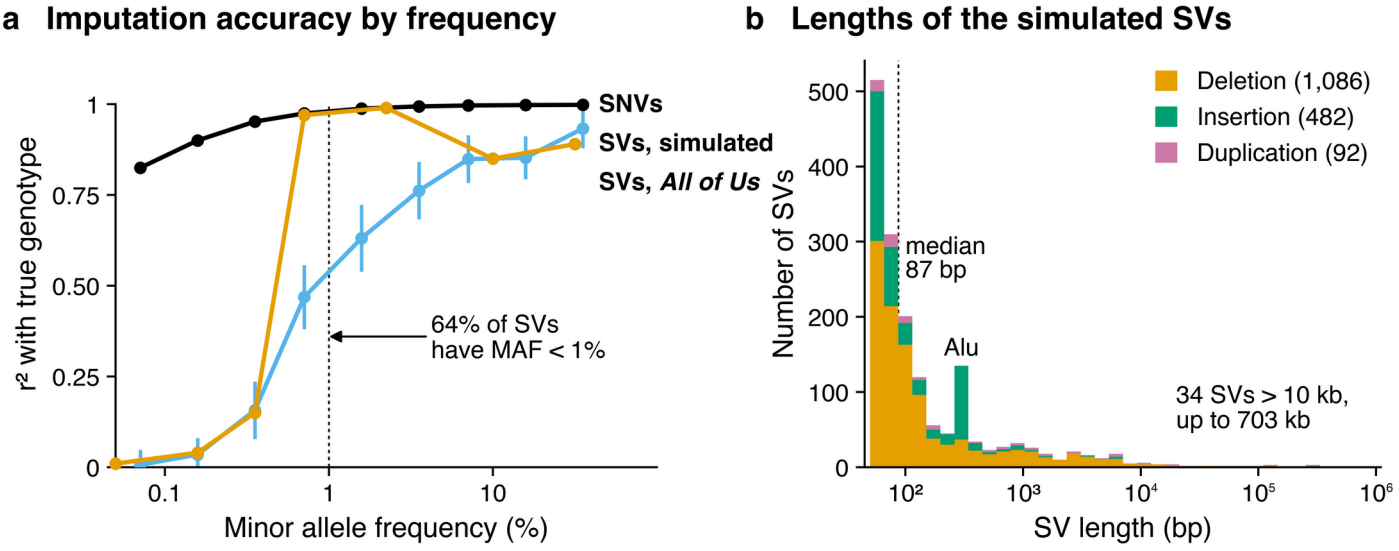


Figure 5 | The simulated SVs are mostly rare and short, and rare SVs impute poorly. a, The r^2 of each variant's measured dosage with its true genotype over all 50,000 simulated individuals (not Beagle's DR2), for 1,662 SVs imputed by Beagle and 6,000 sampled SNVs called from simulated reads on chromosome 22, by minor allele frequency: each bin's median (points) with an approximate 95% interval (1.82 IQR/$\sqrt{n}$, bars), SNVs black and SVs blue, beside the median r^2 of SVs imputed in All of Us per frequency bin (orange, no interval). b, Lengths of the 1,662 measured SVs, stacked by class (records without a type are classed by the sign of their length change). The insertions' peak near 300 bp is where Alu elements lie. bench-sim v7 cohort from public 1000 Genomes haplotypes.

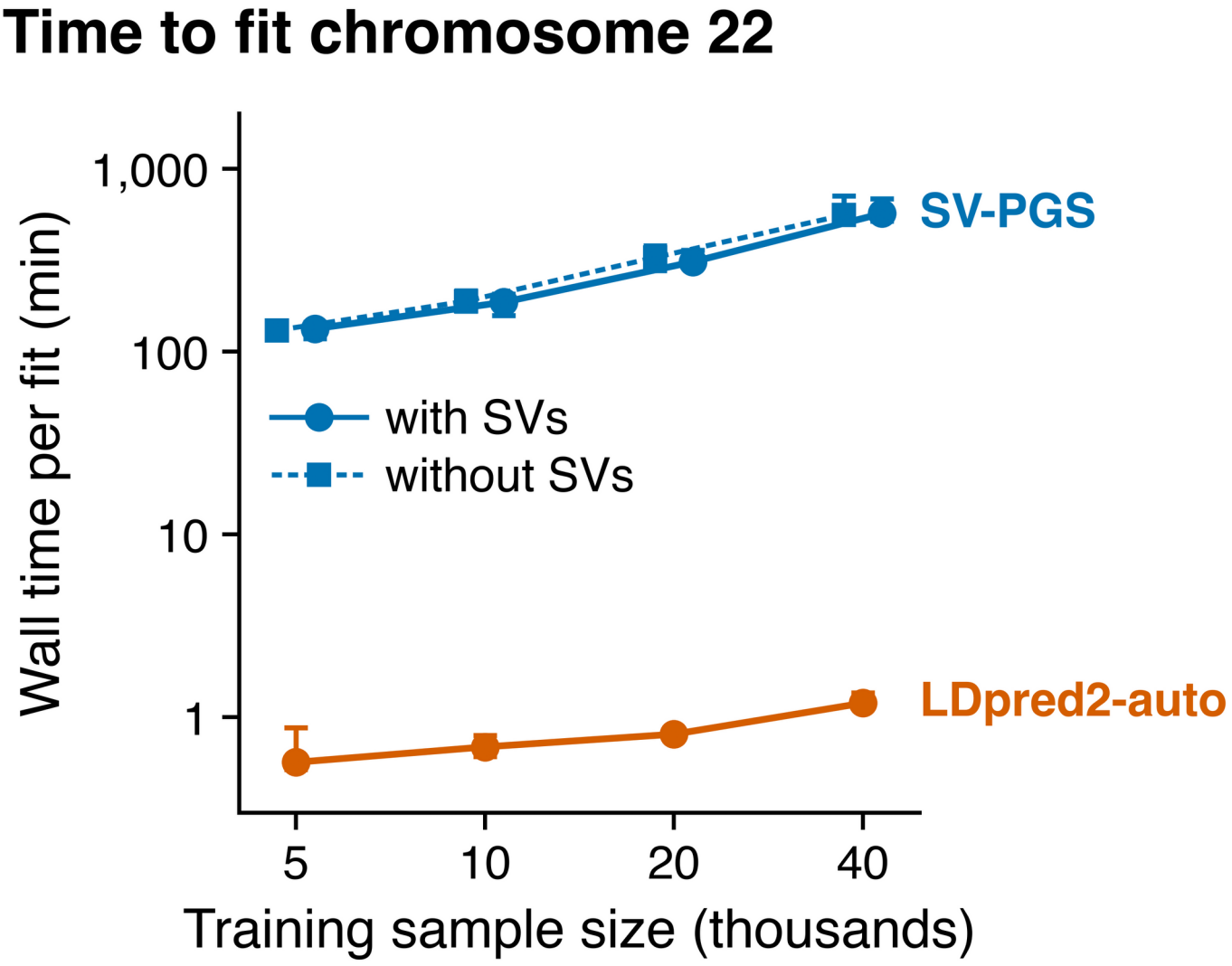


Figure 6 | SV-PGS fits take hours and LDpred2-auto's minutes, and SVs add no time. Wall time of one fit on chromosome 22 per method and training size, pooled over every architecture: median and its distribution-free 95% interval (59-87 fits each, capped bars, most within the marks). MSI Slurm jobs, one fit per job. SV-PGS fits every variant by empirical Bayes (circles and solid line with SVs, squares and dashed line without), LDpred2-auto HapMap3-density SNVs. The results do not record each job's hardware, so the gap compares the methods as they were run, not on equal resources.